

Lightweight Traceability for the Agile Architect

Sebastian Gayer, MID

Andrea Herrmann, Herrmann & Ehrlich

Thorsten Keuler, John Deere

Matthias Riebisch, University of Hamburg

Pablo Oliveira Antonino, Fraunhofer Institute for Experimental Software Engineering

Although traceability information is vital to many software engineering activities, agile development approaches rarely use it because of the extra effort required. In this industry case study, the authors report the results of applying their lightweight semiautomated method that integrates traceability into software architecture developed with an agile process like Scrum.

Agile software development approaches such as Scrum increase the efficiency and flexibility of software development by reducing organizational overhead and emphasizing direct communication instead of comprehensive documentation. All documentation must satisfy a particular need and must be as lightweight as possible. These approaches work well for small collocated collaborative

right balance between reducing process overhead by minimizing documentation and maintaining lightweight development with minimal documentation is difficult to achieve.

Unlike traditional development, in which architecture is a huge upfront design, architecture in agile development must adapt to changing requirements and remain lightweight.⁵ In the agile development world,

teams that meet daily because they can discuss any open issue face to face. Documentation translates to minutes of team meetings, which tends to reduce the architectural metaphor to a sketch.

Nevertheless, the merits of using agile development approaches to create software architecture are still being debated, and agile methods are increasingly being applied in long-term and distributed projects.^{1,2} Such projects often have larger teams, and not all developers know the complete system.³ In a controlled and extended evolution of complex software systems within larger distributed teams, the documentation of an explicit architecture is critical,⁴ but the

architecture is far from monolithic: it is a dynamic communication vehicle that serves as input to subsequent system design and development activities, and agile architects document any assumptions made.⁶ An explicit representation of architecture and traceability serves as documentation to satisfy certification and standards compliance, or it can be the basis for negotiations between customer and provider.

IEEE defines architecture as “the fundamental organization of a system, embodied in its components, their relationships to each other and to the environment, and the principles governing its design and evolution.”⁷ Under this definition, software architecture includes the rationale of decisions and traceability links to other artifacts, such as requirements and test cases.

Traceability between artifacts is a way to support the maintainability, changeability, and sustainability of a software system and its artifacts. Traceability answers diverse questions during software engineering, usually across multiple artifacts; for example, in product-line development, it captures which functionalities have been delivered with which code version.^{8,9} The required information and relationships must be documented and available in a form that supports activities like impact analysis,¹⁰ which identifies the potential changes due to changing requirements before these changes occur.¹¹

The definition of traceability from the Center of Excellence for Software and Systems Traceability (CoEST) states these functions explicitly:¹²

[Traceability is] the ability to inter-relate any uniquely identifiable

software engineering artifact to any other, maintain required links over time, and use the resulting network to answer questions of both the software product and its development process.

To demonstrate the feasibility of using traceability in agile development, we developed a traceability procedure that integrates traceability information into existing artifacts and identifies and updates it semiautomatically, requiring only trivial extensions to existing toolsets. To ascertain the right balance between development and documentation, we applied our approach in an industrial case study—a project at John Deere. The results of the study show that developers can create and maintain architectural documentation of an agile process, such as Scrum, with little additional effort.

WHAT DOES LIGHTWEIGHT MEAN?

To ensure that our approach would be lightweight, we adopted three main strategies:

- › identify the necessary traceability information using a goal-oriented approach,
- › ensure that the creation and maintenance of traceability links can be integrated with existing processes and tool chains, and
- › provide automated support for traceability link creation and maintenance.

We designed our approach to work with any agile software development process, although we use Scrum—generally accepted in the software

industry as a de facto process—in our case study. Figure 1 presents the basic Scrum process and artifacts.¹³ The user stories in each sprint (which refers to the time block in which specified work must be delivered) are detailed into task cards. Architecture is viewed as part of the product and integrated into the Scrum process in that context, for example, as architecture sprints, user stories, and tasks. The code- and version-management system contains developer comments, which can cover traceability information. The lines between artifacts show the three necessary traceability links: requirements to user stories, which drives design decisions; user stories to architecture, which drives implementation; and user stories to test cases, which drives testing.

RETRIEVING TRACEABILITY INFORMATION

Some traceability is implicitly established during the creation of development artifacts and thus does not demand any additional effort. For example, most modeling tools automatically create links such as the ownership relation between two classes, and coding tools indirectly create module interdependencies simply by accessing other code units' interfaces. These naturally occurring links are used in conducting activities like impact and coverage analyses or in understanding how cross-layer elements relate. The challenge is how to retrieve that traceability information from the tools' infrastructure.

Other links require considerably more effort to create and maintain. All vertical traceability links, such as those from requirements to

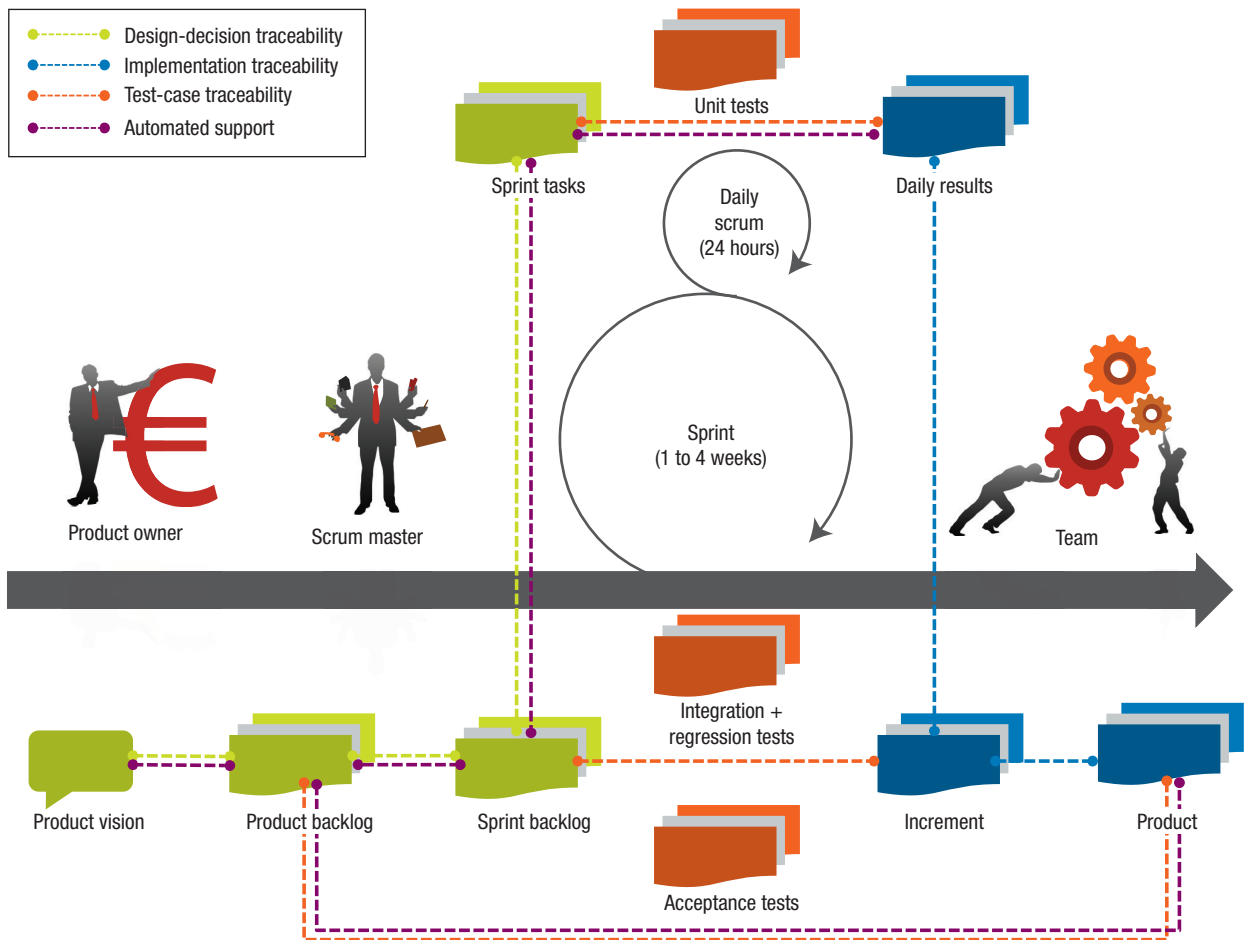


FIGURE 1. Scrum development process extended with traceability links. Our method provides automated support for traceability tasks (purple dashed lines), which include the creation and maintenance of traceability links among user stories listed in the sprint backlog, design decisions, and component implementation for that sprint, and test cases related to those user stories and design decisions.

architecture and from requirements to implementation artifacts, require some degree of effort, depending on the granularity of the interlinked artifacts. Linking high-level artifacts, for example, is much easier than creating links between those at a lower level. Our approach aims to address both types. To address links that require effort to create, we incorporate two steps: analyze needs and adapt the tools' infrastructure accordingly.

Analyze needs

Analysis must be done in combination with the underlying process and tool infrastructure. Needs are usually driven by the risks attached to

the project, which in turn are strongly influenced by the degree of team distribution and system complexity, the quality requirements to be fulfilled, and the need for sustainability.

Adapt tool infrastructure

The minimal information needed to create a link and integrate it into the tool's infrastructure is the trace's source and target. If available, the traceability link type is useful; an example is a structural relation such as part-of, refined-to, or called-by. If possible, the development activity could be added to the link, such as Trac item, bug report, user story, or even a sprint task. Links must be bidirectional, as both directions are

needed for traversal. Examples are coverage checks and comprehension activities for reverse engineering.

ACQUIRING TRACEABILITY LINKS

Acquiring traceability links requires creating either implicit relationships as part of engineering activities or explicit relationships in addition to engineering activities.

Regardless of how they are used, traceability links must be captured and documented somewhere using either a single model or multiple artifacts. Keeping links in a single model makes it hard to connect the links to external artifacts; storing links in

multiple artifacts requires integrating a large number of artifacts and links. Regardless of the capture and document strategy, semiautomatically creating links can reduce the effort involved. Combining semiautomatic link creation with tools to automate link maintenance removes many of the additional costs and effort associated with traceability.

One of the main barriers to ensuring traceability among development artifacts is that people with different project roles and professional backgrounds create those artifacts, and they tend to prefer familiar development tools. Consequently, there is less willingness to migrate to highly integrated tools with built-in traceability support, such as MID's Innovator (www.mid.de/en/products/innovator-enterprise-modeling-suite.htm), VersionOne, and ThoughtWorks' Mingle. This reluctance puts a burden on any traceability approach, which must aim to be as noninvasive on development processes and developer preferences as possible.

MAINTAINING TRACEABILITY LINKS

Agile development's guiding philosophy is to embrace change, which complicates traceability because any development alteration requires updating traceability links. To avoid excess maintenance effort, traceability methods should update links only if necessary, such as to correct an inconsistency with a changed model element.

Developers can easily identify obsolete and erroneous links by comparing the modification dates of the trace's source and target elements and verifying the link if the source element

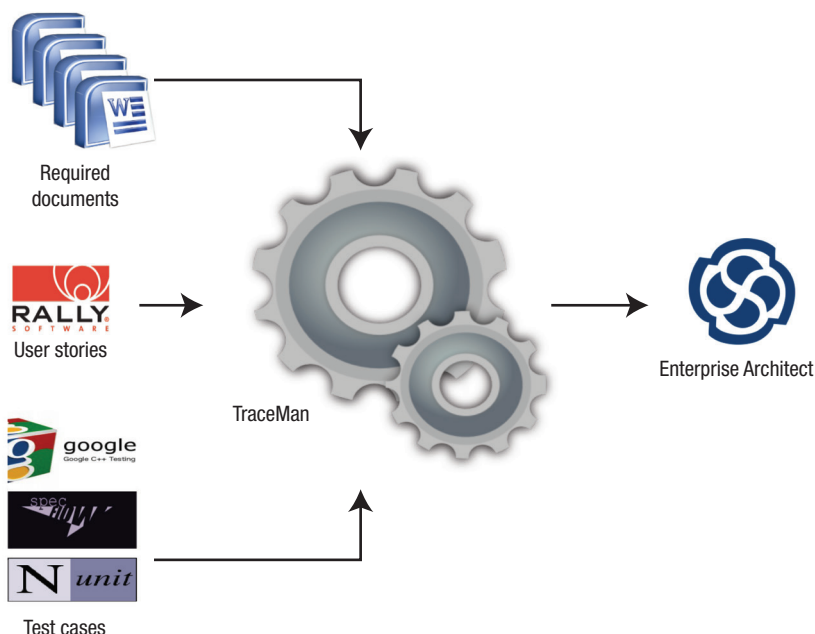


FIGURE 2. Inputs to TraceMan as applied in the John Deere project. TraceMan unified traceability links between requirements, user-story, and test-case formats by providing a centralized repository for the links that served as input to Enterprise Architect. By partially automating link creation with fully automating link maintenance, TraceMan reduced much of the additional cost and effort typically associated with traceability.

has a more recent date. A tool such as TraceMaintainer can simplify this verification by automating the rule-based maintenance of trace links.¹⁴

JOHN DEERE CASE STUDY

To evaluate our approach, we integrated it into the agile development process and tool infrastructure at John Deere.

John Deere decided to introduce agile software development to more quickly respond to customer needs and provide robust solutions as rapidly as possible. The compelling advantages of going agile are short release cycles and early customer feedback; the downside is the inevitable communication overhead associated with a distributed development team.

Project characteristics

We chose to use our approach in a project to design the display for agricultural tractors, which is the machine operator's interface and thus critical to the tractor's success as a product.

The project had the following characteristics:

- › more than 150 developers;
- › global development using Qt, Linux, and C++;
- › a system of 10 million LOC, translating to roughly 100 components;
- › a sprint of 2 weeks; and
- › a potentially shippable increment (PSI) in 8 weeks.

The project's risk factors were distributed teams, team and project size, and the high number of requirements to ensure product quality.

Traceability challenges

At John Deere, user stories are created in Rally (www.rallydev.com), requirements specifications documents and architecture design models are created in Microsoft Word using text editors and a dedicated modeling tool, and test cases are specified with Google's C++ Test Framework, SpecFlow, and NUnit. However, direct links between user stories and requirements specifications were missing, which led to several challenges throughout the process.

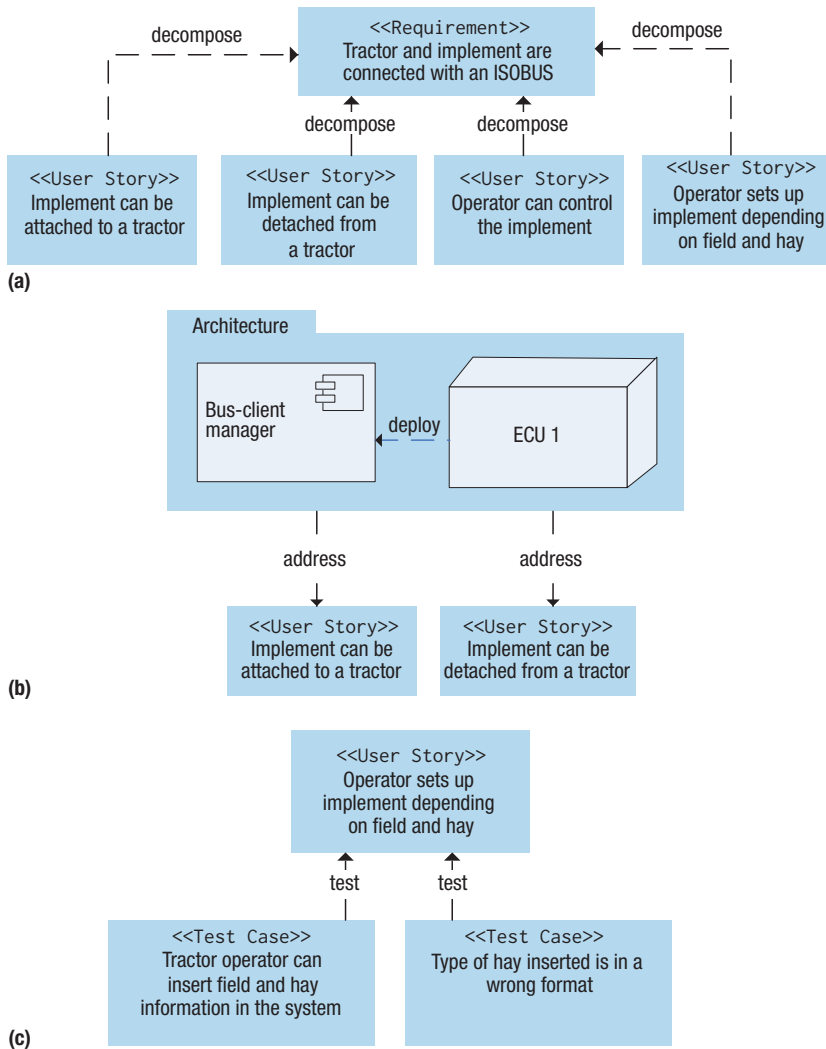


FIGURE 3. Creating traceability links among architectural artifacts in the John Deere project. Links shown are between (a) requirements and user stories, (b) architectural design and user stories, and (c) a user story and two test cases. To support link creation, our approach automatically imports elements, such as user stories and test cases, into a common environment. Developers can then easily and quickly create links manually. ECU: electronic communication unit.

To improve this situation, John Deere included Sparx Systems' Enterprise Architect and Fraunhofer's TraceMan as part of an effort to centralize agile development activities. Enterprise Architect is a well-known and widely adopted modeling tool. TraceMan, developed at the Fraunhofer Institute for Experimental Software Engineering (IESE), is at the core of our traceability procedure. As Figure 2 depicts, it provides automated support in creating traceability links among documents created with

Microsoft Word; user stories created in Rally, and test cases specified in C++ Test Framework, SpecFlow, and NUnit. It also houses the links in a central repository that serves as input to the Enterprise Architect environment.

Establishing links

User stories and requirements specifications were imported to Enterprise Architect with TraceMan, and the traceability links were established and maintained in a centralized model. Within Enterprise Architect,

architectural elements such as interfaces, components, and design patterns were targets of these links. The automatic import function in our tool extensions allowed us to quickly link requirements specifications in Word to user stories in Rally.

In many cases, a project team opts to refactor the source code and architecture incrementally, which enables them to reduce the number of links, retaining only the most relevant ones. Refactoring inherently involves modifying tests and sometimes requirements specifications, so work to update links does not incur any noticeable overhead effort. Our method explicitly addresses refactoring effects on traceability by linking product vision, product backlog, and sprint backlog (see Figure 1).

Figure 3 shows a specific example of link creation. In Figure 3a, the requirement elements "tractor" and "implement" are connected with an ISOBUS (a communication unit within the tractor), which has associated textual documentation specifying the details of that connection and the requirement's decomposition into four user stories. We supported this link-creation step by automatically importing the four user stories created in Rally into the same environment where requirements are specified using model-based approaches.

The link in Figure 3b highlights the importance of connecting user stories to design elements. We have observed that many projects fail to delineate an explicit relationship between architectural design and the elements, such as user stories, that motivated particular design decisions. In this case, the link shows that two specific user stories

TRACE TO LINK REQUIREMENTS AND USER STORIES

- » **Before:** Trace is not done.
- » **After:** Trace takes approximately 2 minutes for each user story.
- » **Assumptions:** Product owners previously created user stories in Rally. Requirements are in Word or Excel files.

Requirements engineers import the user stories from Rally that are associated with the specified requirement. TraceMan creates a modeling element that represents the Rally user story and a model element that represents the requirement. Requirements engineers manually create the trace links between them.

The estimated 2 minutes includes time to type the user story number, for the tools to communicate, to check that the user story was correctly imported, and to create the link.

drove an architectural decision—the inclusion of a bus client manager and electronic communication unit (ECU). We have found that explicit links also reduce the number of unaddressed architecture drivers, as well as the number of architecture elements with no responsibilities.

As with requirements and user stories, our approach automatically imports test cases into the same working environment as the requirements and user stories. In the sample link in Figure 3c, the user story “Operator sets up implement depending on field and hay” is tested by the test cases “Tractor operator should be able to insert field and hay information in the system” and “Type of hay inserted is in a valid format.”

Traceability management

As the various sidebars describing established traces describe, traceability management during the John Deere project was both efficient and effective in large part because product owners and system engineers could work on a centralized model that united user stories specified in Rally and requirements specified in Microsoft Office tools. Traceability links were quickly narrowed to only the most important ones.

Benefits to architecture and architects

Our approach not only simplified architectural decisions because of the central location and structure of the artifacts and their traceability links, but it also supported impact analysis during software evolution. Traceability links can help in predicting how the requirements changes will affect the architecture. Without documented links, it was difficult to trace which architectural

TRACE TO LINK USER STORIES AND ARCHITECTURE

- » **Before:** Trace is not done.
- » **After:** Trace takes less than 10 minutes per user story.
- » **Assumptions:** User stories are in Rally. An architecture model exists.

The product owner and architects discuss which architectural elements are necessary (ones that either exist or need to be created) to address the user story. Once they identify dependencies among the necessary elements, they manually create trace links. Most of the required time is devoted to the discussion to determine the necessary elements.

decisions had to be reconsidered, as the only support came in the form of specification documents and architectural models.

Impact analysis also improved because traceability from the user stories to the architecture enabled

architects and systems engineers to identify exactly which architecture elements had to be considered if the user stories changed.

Sprint planning also became more efficient because, once all the candidate artifacts were identified, it was

TRACE TO LINK USER STORIES AND TEST CASES

- » **Before:** Trace is done ad hoc, taking an average of 30 minutes per user story.
- » **After:** Trace takes less than 10 minutes per user story.
- » **Assumptions:** User stories are in Rally and were imported to the modeling tool with TraceMan. Test cases are in NUnit or SpecFlow test tools.

Testers and product owners discuss which test cases will be used to verify the acceptance criteria associated with a user story. As soon as the testers and product owner identify the necessary tests, the testers use TraceMan to import the test cases created in NUnit or SpecFlow into the modeling tool. The product owner and testers manually create links between the user stories and test-case elements.

Most of the 10 minutes allotted is devoted to deciding if the test suite will verify the criteria. Once the test suite is identified, TraceMan is used to create traceability links.

easier to estimate the effort to implement each feature and select the most suitable artifact for the sprint.

Improvements in test specifications

Each user story has a set of acceptance criteria that should be reflected in test specifications. In John Deere's typical process, user stories are documented in Rally, and test specifications are stored in the databases of the tools that developed them. When a user story changed, it took considerable effort to manually verify if the associated test was still valid. With our approach, test specifications are imported into the modeling tool. As user stories are imported from Rally into the modeling tool, the traces are created with automated support, making the subsequent discussion more fruitful.

It is also important to highlight which user story is related to which code and whether that code has been tested, reviewed, and working. Also important are the connections between sprint tasks and daily results and the unit tests

to verify the validity of those results and between product backlog and product and the acceptance tests that verify the validity of the produced results.

As with the traces between requirements and user stories, the traces between test cases and user stories are created manually. Again, extra effort is minimal because the agile development team can easily trace the elements while they discuss which test cases are needed to completely cover each user story.

Effects on the development environment

Our approach gave John Deere developers a structured way to communicate requirements among stakeholder groups—perhaps the most valuable benefit of the case study project. With the resulting centralized information source, engineers could combine elements from agile development and traditional document-centric development, presenting an opportunity for the company to enhance its overall development process with reasonable effort.

We have presented a procedure with automated support for creating and maintaining a lightweight traceability-management solution for agile software development. To minimize communication overhead, our approach relies on extensions to agile artifacts and popular tools. Link identification is semi-automatic, and links can be visualized graphically within existing tools.

Our case study in an industrial project demonstrated the feasibility of implementing our approach in a mainstream model-based development environment with only minor extensions. Any agile software development process can be extended similar to the way we extended Scrum.

In future work, we plan to increase the degree of automated support for creating traceability links using connections in impact analysis, and applying rule-based consistency checks between related artifacts during software evolution. ■

ACKNOWLEDGMENTS

We thank members of the German Computer Society's Special Interest Group on Traceability—Elke Bouillon, Baris Güldali, Steffen Lehnert, and Daniel Moldt—for their fruitful discussions about early drafts of this article.

REFERENCES

1. J. Eckstein, *Agile Software Development in the Large—Diving Into the Deep*, Dorset House, 2004.
2. C. Larman and B. Vodde, *Practices for Scaling Lean & Agile Development: Large, Multisite, & Offshore Product Development with Large-Scale Scrum*, Addison-Wesley, 2008.
3. L. Cao and B. Ramesh, "Agile Requirements Engineering Practices:

- An Empirical Study," *IEEE Software*, vol. 25, no. 1, 2008, pp. 60–67.
4. P. Kruchten, "Architecture," blog, Kruchten Engineering Services, 2014; <http://philippe.kruchten.com/architecture>.
 5. P. Kruchten, "Software Architecture and Agile Software Development: A Clash of Two Cultures," *Proc. ACM Int'l Conf. Software Eng. (ICSE 10)*, 2010, pp. 497–498.
 6. D. Falesi et al., "Peaceful Coexistence: Agile Developer Perspectives on Software Architecture," *IEEE Software*, vol. 27, no. 2, 2010, pp. 23–25.
 7. *IEEE Std. 1471-2000, IEEE Recommended Practice for Architectural Description for Software-Intensive Systems*, IEEE, 2000; <https://standards.ieee.org/findstds/standard/1471-2000.html>.
 8. S. Rochimah, W. Wan Kadir, and A.H. Abdullah, "An Evaluation of Traceability Approaches to Support Software Evolution," *Proc. 2nd IEEE Int'l Conf. Software Eng. Advances (ICSEA 07)*, 2007, pp. 19–27.
 9. N. Aizenbud-Reshef et al., "Model Traceability," *IBM Systems J.*, vol. 45, no. 3, 2006, pp. 515–526.
 10. A. De Lucia, F. Fasano, and R. Oliveto, "Traceability Management for Impact Analysis," *Proc. ACM Frontiers Software Maintenance (FoSM 08)*, 2008, pp. 21–30.
 11. S. Winkler and J. von Pilgrim, "A Survey of Traceability in Requirements Engineering and Model-Driven Development," *Software and Systems Modeling*, vol. 9, no. 4, 2010, pp. 529–565.
 12. J. Cleland-Huang et al., "Software Traceability: Trends and Future Directions," *Proc. ACM Future of Software Eng. Conf. (FoSE 14)*, 2014, pp. 55–69.

ABOUT THE AUTHORS

SEBASTIAN GAYER is a managing consultant at MID GmbH. His research interests include the Innovator modeling platform, model-driven software development, requirements engineering, enterprise-architecture management, and service-oriented architecture. Gayer received a Diploma in information engineering and management from the Karlsruhe Institute of Technology. Contact him at s.gayer@mid.de.

ANDREA HERRMANN is a freelance software-engineering trainer and researcher with Herrmann-Ehrlich. Her research interests include requirements engineering, decision making in software engineering, and empirical research. Herrmann received a PhD in physics from the University of Stuttgart. She is a member of the German Informatics Society and a member and leader of several working groups of the German chapter of ACM. Contact her at herrmann@herrmann-ehrllich.de.

THORSTEN KEULER is a product engineering supervisor of the John Deere Intelligent Solutions Group in Kaiserslautern. His research interests include quality improvement processes and optimized sustainable measures to support the delivery of high-quality software. Keuler received a PhD in computer science from the Technical University of Kaiserslautern. Contact him at KeulerThorsten@JohnDeere.com.

MATTHIAS RIEBISCH is a professor of software engineering at the University of Hamburg. His research interests include software evolution and sustainability, model-based development, and software architectural design methods. Riebisch received a PhD in computer science from the Ilmenau University of Technology. He is a member of the German Informatics Society and cochair of its Software Architectures group. Contact him at riebisch@informatik.uni-hamburg.de.

PABLO OLIVEIRA ANTONINO is a senior engineer and project manager at the Fraunhofer Institute for Experimental Software Engineering, Kaiserslautern. His research interests include the construction and analysis of architectures of dependable embedded systems to support automotive, agricultural, medical-device, and avionics applications. Antonino received a PhD in computer science from the University of Kaiserslautern. Contact him at pablo.antonino@iese.fraunhofer.de.

13. K. Schwaber, *Agile Project Management with SCRUM*, Microsoft Press, 2004.
14. P. Mäder, O. Gotel, and I. Philippow, "Rule-Based Maintenance of Post-Requirements Traceability Relations," *Proc. 16th IEEE Int'l Conf. Requirements Eng. (RE 08)*, 2008, pp. 23–32.



Selected CS articles and columns are also available for free at <http://ComputingNow.computer.org>.

